

PREREGISTRO_FASE_2

QuantDialectics — librería del Protocolo Empírico Trinitario

José Mauricio Gómez Julián

2026-08-20

Índice

PRE-REGISTRO — FASE 2, las elementales de dos niveles y la escalada	1
0. Lo que el paso 0 dejo medido, y que este pre-registro toma como dado	1
1. Que se construye, y con que contrato	2
2. Las cinco clases de monotonía, declaradas ANTES de medirlas	2
3. Los tres puntos donde el valor extremo es EXACTO y no lleva holgura	2
4. La búsqueda de puntos críticos, y por que es segura en la dirección correcta	3
§4-bis. ENMIENDA, escrita antes de correr nada y antes de la primera línea de la pieza	3
5. Decoraciones: la regla local de las elementales NO es la de la aritmética	4
6. Los controles positivos, escritos ANTES de la pieza	4
7. PORTON 2 — el barrido de aceptación, con sus criterios escritos	4
§7-bis. ENMIENDA, escrita al ver el primer barrido, y por que NO es reinterpretar	5
8. Lo que este pre-registro NO promete	6

PRE-REGISTRO — FASE 2, las elementales de dos niveles y la escalada

Escrito ANTES de la primera línea de `R/elementary.R` y antes de correr el barrido.
Sesión 80. Lo que sigue no se reinterpreta al ver los números: si un criterio falla, falla, y el diagnóstico se ancla en datos ANTERIORES a este archivo.

0. Lo que el paso 0 dejo medido, y que este pre-registro toma como dado

- Suite de arranque: **PASS 330 / FAIL 0 / WARN 0 / SKIP 2 / ERROR 0**, la cifra de la 79.
- `ra_verify_operator_table()` da `closed = TRUE` con los cuatro criterios vacíos, y su control positivo dispara (admitir γ a la fuerza hace escapar digamma , psigamma , trigamma).

- **Las 16 cifras C3 re-auditadas contra la fuente primaria**, no contra la ficha: reporte de Gladman, Innocente, Mather, Ozaki y Zimmermann, edicion de **febrero de 2026**, Tabla 3, columna **GNU libc 2.43**. Las 16 coinciden digito a digito. La fuente quedo depositada en Robust Arithmetic/Literatura/ (no estaba: la 79 la leyo y no la bajo).
- **Un dato del §3 y del §3.1 del reporte que la bitacora no tenia y que la fase 2 necesita**: la busqueda recorre el espacio ENTERO de binary64 mapeado a uint64_t, y el propio texto nombra el caso de la reduccion de argumento cerca de 2^{1024} para seno y coseno como algo que el algoritmo debe detectar solo. Es decir: $e(\sin)$ y $e(\cos)$ no son cifras de un tramo chico de argumentos, sino cotas inferiores sobre todo el rango. Sin ese dato, la holgura de $\sin/\cos$ tendria un agujero justamente donde la fase 2 la apoya.

1. Que se construye, y con que contrato

`ra_elem(fun, x, level, precision)` devuelve un `ra_ivl` que **encierra** el conjunto $\{ f(t) : t \text{ en } X, t \text{ en } \text{dom}(f) \}$. El modo de falla es unilateral: sobreestima, nunca miente.

Nivel rapido (`level = "fast"`, sin dependencias): evaluacion de la libm en los extremos correctos por monotonia por tramos, ensanchada `ra_slack(f)` pasos hacia afuera por extremo.

Nivel riguroso (`level = "rigorous"`, via `Rmpfr`): evaluacion a `precision` bits en `nearest`, `roundMpfR(., 53, D/U)` y `toNum(., D/U)` — el puente medido, `as.numeric` nunca.

2. Las cinco clases de monotonia, declaradas ANTES de medirlas

clase	funciones	regla del rango sobre $[a,b]$
creciente	<code>exp log log2 log10 loglp</code> <code>expml sqrt sinh tanh asin</code> <code>atan</code>	$[f(a), f(b)]$
decreciente	<code>acos</code>	$[f(b), f(a)]$
valle en 0	<code>cosh</code>	$lo = 1$ si $0 \text{ en } [a,b]$, si no $\min(f(a), f(b))$; $hi = \max(f(a), f(b))$
periodica	<code>sin cos</code>	busqueda de puntos criticos con <code>pi INTERVALAR</code>
con polos	<code>tan</code>	busqueda de polos con <code>pi INTERVALAR</code>

Control positivo de esta tabla (obligatorio, se escribe antes): la clase de cada funcion NO se declara a mano y se cree; se **verifica** contra la propia funcion sobre una malla, y una clase puesta mal a proposito DEBE hacer fallar el arnes.

3. Los tres puntos donde el valor extremo es EXACTO y no lleva holgura

$\sup \sin = \sup \cos = 1$, $\inf \sin = \inf \cos = -1$, $\inf \cosh = 1$. Los tres son enteros representables y son el valor matematico exacto del supremo, no una evaluacion de la libm. **Aplicarles holgura seria ensanchar sin razon**; no aplicarsela es correcto por teorema. `sqrt(0) = 0` cae en la misma categoria.

4. La búsqueda de puntos críticos, y por que es segura en la dirección correcta

Para $\sin$ sobre $[a, b]$ con $a < b$: los máximos están en $\pi/2 + 2k\pi$ y los mínimos en $3\pi/2 + 2k\pi$. Se computa $K = (X - \pi/2) / (2\pi)$ con **aritmética intervalar y π encerrado**, y se pregunta si K contiene un entero ($\text{floor}(\sup K) \geq \text{ceiling}(\inf K)$).

K es un SUPERCONJUNTO del conjunto verdadero de índices. Por lo tanto:

- si el intervalo verdadero contiene un extremo, K lo detecta **siempre** — no hay falsos negativos, que es el único modo de falla que rompería la contención;
- K puede detectar uno que no está — falso positivo, que ensancha y no miente.

La consecuencia declarada, con su precio: cuando $|a|$ es grande, K se ensancha hasta contener enteros siempre, y $\sin(X)$ sale $[-1, 1]$. Es un encierro válido y flojo. **La frontera se pre-registra como magnitud, no como impresión:** se mide a partir de que exponente de a el nivel rápido deja de resolver $\sin(X)$ mas fino que $[-1, 1]$, y esa cifra se publica.

§4-bis. ENMIENDA, escrita antes de correr nada y antes de la primera línea de la pieza

Al derivar la aritmética de K para escribir la pieza — no al ver un número — me di cuenta de que **la frase de arriba le atribuye la frontera a la causa equivocada**, y la dejo escrita con su corrección en vez de reescribirla, porque el pre-registro se enmienda a la vista y no se tapa.

$K = X/(2\pi) - q/m$ tiene dos fuentes de ancho: el ancho del propio X dividido por 2π , y el error de redondeo de la división. Sea w el ancho de X y a su magnitud. La primera fuente aporta $w/(2\pi)$. La segunda aporta del orden de $|K| \cdot 2^{-52} = (a/(2\pi)) \cdot 2^{-52}$, es decir $(a \cdot 2^{-52})/(2\pi)$. Pero **$a \cdot 2^{-52}$ es exactamente un ulp de a , y ningún intervalo no degenerado puede ser mas angosto que un ulp:** $w \geq a \cdot 2^{-52}$. Por lo tanto la segunda fuente **nunca supera a la primera por mas de un factor chico**, y la precisión de la reducción **no es lo que manda**.

Lo que manda es la **granularidad de binary64**: K contiene un entero en cuanto $w \geq 2\pi$, y como $w \geq \text{ulp}(a)$, todo intervalo no degenerado con $\text{ulp}(a) \geq 2\pi$ — esto es, $|a| \geq 2^{52} \cdot 2\pi \sim 2^{54,65}$ — DEBE dar $[-1, 1]$, y esa respuesta es la **correcta y estrecha**, no una pérdida: dos dobles vecinos de esa magnitud están a mas de un periodo uno de otro y el seno recorre su rango entero entre ellos.

Lo que esto cambia en el pre-registro, y es una predicción falsable:

1. La frontera medida para un ancho RELATIVO r debe caer en $e^* = \text{ceiling}(\log_2(2\pi / r))$, la cifra de granularidad, **y no en un exponente gobernado por 2^{-53}** . Se pre-registra: $|\text{e_medido} - e^*| \leq 1$ para los dos niveles.
2. **El nivel riguroso NO debe correr la frontera de manera apreciable.** Se pre-registra $\text{e_riguroso} - \text{e_rapido}$ en $\{0, 1\}$. Si el riguroso la corriera mucho, la derivación de arriba sería falsa y habría que rehacerla; si el rápido cayera muy por debajo del riguroso, la reducción rápida estaría peor de lo que esta derivación admite. **El criterio puede fallar por los dos lados**, que es la única forma en que un criterio informa.
3. La frontera se mide con ancho relativo y no absoluto, porque un ancho absoluto fijo se vuelve degenerado ($a + w \approx a$) mucho antes de llegar a la frontera y no puede alcanzarla.

Y queda dicho lo que la enmienda NO toca: la contención. Falsos negativos de la búsqueda de extremos serían el único modo de romperla, y no hay ninguno por construcción (K es superconjunto). Los criterios A y B del §7 quedan exactamente como estaban.

5. Decoraciones: la regla local de las elementales NO es la de la aritmetica

`.ra_local_total()` mira solo los extremos de SALIDA, y para las elementales eso es insuficiente en un caso real: `atan([-Inf, Inf])` sale acotado desde una entrada no acotada, y la clausula 5.5.1 pide que `com` afirme que **la caja de entrada** es acotada. Se escribe una regla local propia:

- `trv` si X no esta contenido en $\text{dom}(f)$ (falla `def`) o si X es vacio;
- `dac` si X esta en $\text{dom}(f)$ pero X o el resultado no son acotados;
- `com` si X esta en $\text{dom}(f)$, X es acotado y el resultado es acotado.

Y se mete con las decoraciones de entrada por la regla del minimo (5.6). **Errores de dominio son `trv`, nunca `ill`** — la correccion del texto que la 79 documento.

6. Los controles positivos, escritos ANTES de la pieza

CP-1 (el del prompt). Un punto donde la libm difiere del correctamente redondeado — buscado con `Rmpfr` en la malla, no inventado — DEBE quedar dentro del enclosure rapido. **Falla el arnes si no se encuentra ninguno:** que la libm sea perfecta en toda la malla seria o un milagro o un medidor roto.

CP-2 (el simetrico, el que prueba que el arnes puede fallar). Una holgura puesta a 0 adrede DEBE producir al menos una violacion de contencion sobre los mismos datos donde el paquete verdadero no produce ninguna. Si con holgura 0 tampoco falla, el barrido no esta tocando ningun punto mal redondeado y el porton 2 seria vacuo.

CP-3 (clase de monotonia equivocada). Declarar `cosh` como creciente DEBE romper la contencion sobre un intervalo que cruza el 0.

CP-4 (la busqueda de puntos criticos apagada). Ignorar los extremos interiores de `sin` DEBE romper la contencion sobre $[0, \pi]$ — donde el maximo es interior y los dos extremos valen casi 0.

CP-5 (el polo de `tan`). `tan([1, 2])` contiene $\pi/2$ y DEBE dar el entero con `trv`; tratarlo como monotono daria `[tan(1), tan(2)] = [1.55, -2.18]`, un par invertido — y el constructor lo rechaza, que es la senal.

CP-6 (dominio). `log([-1, 2])` DEBE dar $[-\text{Inf}, \log 2]$ con `trv`; `log([-2, -1])` DEBE dar el vacio con `trv`; `asin([-2, 2])` DEBE dar $[-\pi/2, \pi/2]$ con `trv` y no un `NaN`.

CP-7 (la escalada). Una decision que el nivel rapido NO puede tomar DEBE resolverse al subir la escalera, y una que no se resuelve ni en 848 bits DEBE salir con la palabra «sin veredicto a este presupuesto (848 bits)» y no con un silencio ni con un veredicto forzado.

7. PORTON 2 — el barrido de aceptacion, con sus criterios escritos

Diseno. Por cada una de las 16 funciones, **10^6 puntos** repartidos por franjas de exponente dentro del dominio de la funcion, mas puntos incomodos inyectados (los bordes del dominio, los ceros, las potencias de dos, los multiples de $\pi/2$ para las trigonometricas). Referente externo: **MPFR a 300 bits**, que no comparte con el nivel rapido ni codigo, ni convencion de redondeo, ni representacion.

Criterio A — contencion, absoluto. `f_MPFR(x)` DENTRO de `ra_elem(f, [x,x], "fast")` para los 10^6 puntos de cada funcion. **Cero salidas.** Una sola salida no agranda la holgura: se investiga.

Criterio B — contencion sobre intervalos no degenerados, absoluto. Por funcion, 10^4 intervalos aleatorios; por intervalo, 100 puntos internos; `f_MPFR` de cada uno DENTRO del enclosure. **Cero salidas.** Esto es lo que prueba la monotonia por tramos y la busqueda de extremos interiores; el criterio A solo prueba la holgura.

Criterio C — el margen, reportado y NO enmascarado. Un porton que pasa sin decir por cuanto no distingue una holgura ajustada de una absurda. Se mide, por funcion, el error real en ulps de la libm sobre la malla:

`e_obs(f) = max sobre la malla de |f_libm(x) - f_MPFR(x)| / ulp(f_MPFR(x))`

y se publica junto a `e(f)` del reporte y a `slack(f)`. **Se pre-registran tres lecturas:**

1. `e_obs(f) <= slack(f)` para las 16 — si no, el criterio A ya habria fallado; es la misma cosa dicha en ulps, y sirve para ver **cuanto** de la holgura se uso.
2. `e_obs(f) <= e(f)` es lo ESPERADO pero **no** es un criterio de aprobacion: `e(f)` es cota inferior hallada por busqueda con umbral $t = 10^6$ sobre 2^{64} entradas, y 10^6 puntos aleatorios pueden razonablemente hallar menos, y con suerte hallar mas. Si sale `e_obs(f) > e(f)`, **es un hallazgo sobre esta maquina y se reporta como tal**, no un fallo.
3. Se publica `e_obs(f) / slack(f)` por funcion. **Si para alguna funcion ese cociente pasa de 0,80, la holgura queda declarada AJUSTADA en la bitacora** — no se toca, se anota, porque agrandarla al ver el numero es exactamente lo que el pre-registro prohíbe.

Criterio D — el nivel riguroso encierra mas fino que el rapido. Por funcion, sobre 10^3 puntos, el ancho del enclosure riguroso a 106 bits DEBE ser menor o igual que el del rapido. Si no lo es en alguna funcion, la escalada no sirve para nada en esa funcion y hay que decir por que.

§7-bis. ENMIENDA, escrita al ver el primer barrido, y por que NO es reinterpretar

El §7 lectura 2 decia: `e_obs(f) <= e(f)` es lo esperado pero no es criterio de aprobacion, y si sale al revés **es un hallazgo sobre esta maquina y se reporta como tal**. Salio al revés para seis funciones, y lo que sigue es el hallazgo, no un aflojamiento del criterio: el criterio de aprobacion (`e_obs <= slack`) se cumple con holgura en las dieciseis y no se toco.

Lo medido, en el barrido chico de 20 000 puntos por funcion: exp 1,279 contra 0,511 publicado; cos 1,184 contra 0,516; expm1 0,996 contra 0,913; sin 0,857 contra 0,516; tan y atan 0,662 contra 0,619 y 0,523. Que 20 000 puntos al azar le ganen a una busqueda dirigida sobre 2^{64} entradas con $t = 10^6$ **es implausible**, y esa implausibilidad es lo que obligo a investigar en vez de anotar.

La causa, aislada y verificada fuera de R. Un programa en C compilado con gcc -02, que lee el argumento de la linea de comandos para que no haya plegado de constantes, llama a `sin(0x1.b981319337b63p-26)` y obtiene `0x1.b981319337b62p-26`, que es el correctamente redondeado (verificado en MPFR y por serie de Taylor a 500 bits, que coinciden a $4e-34$ ulp). **R devuelve 0x1.b981319337b63p-26**, un ulp arriba. Es decir: R y la libm escalar de glibc 2.43 no dan lo mismo, y el que se aparta del correctamente redondeado es R.

Y la causa de la causa, que corrige mi primer diagnostico. Mi primera hipotesis fue «vector de largo ≥ 2 va a `libmvec`»; **es falsa**, y la deje escrita porque tape un diagnostico equivocado seria peor que haberlo tenido. La medicion con `R_ENABLE_JIT=0` la descarta: con el compilador de byte-code apagado, **todos** los caminos dan `...b63`, incluido el de largo 1. Con el JIT en su valor por defecto, el unico camino que da `...b62` es una llamada a `sin` sobre un vector de largo 1 **dentro de codigo compilado**, que es un atajo escalar del compilador de byte-code.

O sea: la diferencia no es de largo de vector, es de **ruta de evaluacion**, y depende de si el codigo esta compilado.

Que se hizo, y que NO se hizo.

- **No** se agrando ninguna holgura. Las 16 quedan exactamente como las declaro la 79.
- **No** se escribio codigo con la forma que dispara el atajo del compilador. Seria depender de un interno no documentado, que es la clase de dependencia que deja de valer sin avisar; y ademas el atajo no se alcanza desde `vapply` ni desde `do.call`, medido.
- **Si** se corrigio el ANCLA declarada. El criterio C3 decia «existe error maximo conocido en ulps para la libm de esta maquina»; queda dicho que esa cifra es un ancla para una **rutina**, no un certificado para la **ruta** que R usa, y que la ruta se **mide**. `ra_measure_library_error()` es esa medicion, exportada, y el porton asiente que la holgura declarada la cubre. Es la regla de apuntar el instrumento a su propio producto: el paquete venia comparandose contra el reporte y nunca contra su propia ruta de evaluacion.
- **Si** quedo asentado, como test permanente, que las dos rutas difieren. Si algun dia dejaran de diferir, el test cae y el comentario que explica el diseno no se vuelve obsoleto en silencio.

Lo que esto le cuesta al margen, dicho sin adornos: la holgura $2e+1$ se penso para absorber que **e** es cota inferior y que el reporte mide otra compilacion del mismo codigo. Ahora absorbe ademas una ruta distinta. El cociente `e_obs/slack` medido es a lo sumo 0,43 en el barrido chico; el barrido de 10^6 lo publica de nuevo y el umbral de 0,80 del §7 lectura 3 sigue en pie sin cambiar.

8. Lo que este pre-registro NO promete

- No promete que el nivel rapido sea **estrecho** en las trigonometricas de argumento grande. Promete que es **valido** y que la frontera de resolucion queda medida y publicada (§4).
- No promete cerrar **tan** cerca del polo con ancho finito: alli el rango es genuinamente no acotado y la respuesta correcta es el entero con `trv`.
- No promete que `e_obs(f)` reproduzca `e(f)`. Ver §7, lectura 2.